

ECE 220 Computer Systems & Programming

Lecture 11 — Problem Solving with Pointers and Arrays

September 29, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

The Finding Problem

**You know pointers, arrays, functions.
Time to solve problems: sort, search.**



Sorted data turns a thousand probes into ten.

Today's Two Questions

Every probe should kill half the possibilities.

1. How fast can you *find* something in an array; what if the array is sorted?
2. How do the classic sorts move data?

Warm-up: Transpose, Flat

Transpose an $M \times N$ matrix into an $N \times M$ one — both stored **flat** (as `int *`).

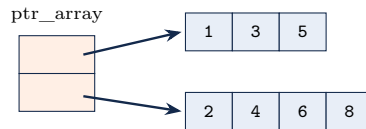
- `in[i][j]` is flat
`in_matrix[i*N + j]`
- It should be copied into
`out_matrix[j*M + i]`
- Why must the callee be told the shape? a bare `int *` has no idea — 2-D shape information is lost at the call

```
void transpose(int *in_matrix, int *out_matrix) {  
    int i, j;  
    for (i = 0; i < M; i++)  
        for (j = 0; j < N; j++)  
            out_matrix[j * M + i] = in_matrix[i *  
                N + j];  
}
```

3 × 4 input 1..12 → first output row 1 5 9

Pointer Array vs. Pointer to an Array

```
int a[3] = {1,3,5};  
int b[4] = {2,4,6,8};  
  
int *ptr_array[2];    /* 2 pointers */  
ptr_array[0] = a;  
ptr_array[1] = b;
```



- `ptr_array[1][2] = 6`
- Rows may have *different* lengths — unlike `int m[2][4]`

An array of addresses — each element is itself a pointer you can index through. The gateway to argv and to ragged data.

Linear vs. Binary Search

Linear: check from the start until found — works on any array.

Binary (sorted arrays only): probe the middle, keep the half that can contain the item.

```
int binary_search(int array[], int n, int item);
```

The array arrives as a pointer, so the length `n` must be passed in. Returns the index of `item`, or `-1`.

Step	Live region	Middle / action
search 23 in	2 5 8 12 16 23 38 56 72 91	
probe 1	indices 0–9	mid = <u>16</u> ; $23 > 16$: <u>second half</u>
probe 2	indices 5–9	mid = <u>56</u> ; $23 < 56$: <u>first half</u>
probe 3	indices 5–6	mid = <u>23</u> — <u>found, index 5</u>

In-class: 1000 sorted elements — how many probes at most? about 10 ($2^{10} = 1024$)

In-Class Problem: Iterative Binary Search

Return the item's index, or -1 .

- What do `low` and `high` mean?
they bracket the live region
- Loop while `low < high`
- Why does it terminate? each probe shrinks `high-low` — strictly

```
int binary_search(int array[], int n, int item) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (array[mid] == item)
            return mid;
        if (item < array[mid])
            high = mid - 1;    /* first half */
        else
            low = mid + 1;    /* second half */
    }
    return -1;
}
```

Bubble Sort

```
void bubble_sort(int array[], int n);
```

One **pass**: compare each pair of neighbors, swap when out of order. Repeat passes until a pass makes **no swaps**.

8	1	4	2	5
1	8	4	2	5
1	4	8	2	5
1	4	2	8	5
1	4	2	5	8

(after pass 1 — 8 has “bubbled up” all the way)

After pass k , the largest k elements are in place.

```
void bubble_sort(int array[], int n) {  
    int i, swapped;  
    do {  
        swapped = 0;  
        for (i = 0; i < n - 1; i++)  
            if (array[i] > array[i + 1]) {  
                swap(&array[i], &array[i + 1]);  
                swapped = 1;  
            }  
    } while (swapped);  
}
```

Note `swap(&array[i], &array[i+1])` — Lecture 9’s pointer swap, earning its keep.

Insertion Sort

Like sorting cards in your hand:

Take	Array
start	5 2 6 1 3 9
2	2 5 6 1 3 9
6	2 5 6 1 3 9
1	1 2 5 6 3 9

(sorted part in bold)

- After step k , the first k elements are sorted among themselves
- Great on nearly-sorted data — each item moves only as far as it must

```
void insertion_sort(int array[], int n) {
    int s, u, temp, empty_idx;
    for (u = 1; u < n; u++) {
        temp = array[u];      /* take next item */
        empty_idx = u;
        for (s = u - 1; s >= 0; s--)
            if (temp < array[s]) {
                array[s + 1] = array[s]; /* shift */
                empty_idx = s;
            }
        array[empty_idx] = temp; /* insert */
    }
}
```

`empty_idx` tracks the gap left by the item we took, so the insert at the end lands in the right place.

Insertion Sort: the Code

```
void insertion_sort(int array[], int n);
```

Grow a sorted prefix, one item at a time.

- u marks the first *unsorted* item: take `array[u]`; the gap starts at u
- s scans the *sorted* part right-to-left; shift each larger element one slot right
- Drop the taken item into the gap

Worst case for n items: about $n^2/2$ shifts, on reverse-sorted input.

The inner loop keeps scanning after the shifts stop — harmless, since the prefix is sorted, but a `break` would save work.

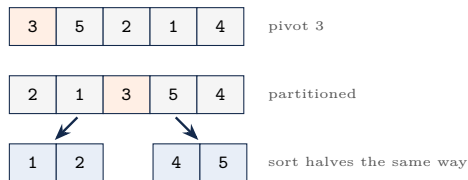
```
void insertion_sort(int array[], int n) {
    int s, u, temp, empty_idx;
    for (u = 1; u < n; u++) {
        temp = array[u];      /* take next item */
        empty_idx = u;
        for (s = u - 1; s >= 0; s--)
            if (temp < array[s]) {
                array[s + 1] = array[s]; /* shift */
                empty_idx = s;
            }
        array[empty_idx] = temp; /* insert */
    }
}
```

Quick Sort (a Preview)

Divide and conquer:

1. Pick a **pivot**; **partition**: smaller elements left, larger right
2. The pivot is now in its final place
3. Sort each side *by the same method*

“Sort each side by the same method” —
quicksort calls *itself*. That idea gets a whole
lecture: recursion, after the midterm.



Summary & What's Next

Today

- **Flat matrices:** shape is yours to track — $i*N+j$ in, $j*M+i$ out
- **Searching:** linear works anywhere; binary needs sorted data and kills half the region per probe
- **Sorting:** bubble (neighbor swaps until a clean pass), insertion (grow a sorted prefix), quicksort (partition around a pivot — and call yourself)

Next

- **Midterm 1 is Thursday, October 1** — L1-6
- Next lecture (Oct 6): recursion — functions that call themselves

Code from today: `transpose.c`, `bsearch.c`, `bubble.c`, `insertion.c`.